AR Sudoku: Vision and Solver Design

Leonardo Tisato

1. Scope

The user points the camera at a printed Sudoku. The app checks that a solution exists and then "writes" the missing digits on the paper one at a time. The digits stay on the paper while it moves.

- **My part (vision + solver):** from a camera frame to grid position, digits read, solution, and the overlay drawn on the frame.
- **Ali's part (app):** camera access, UI, reveal timing, messages to the user, packaging and deployment.

2. Interface

A Python package `sudoku_cv` (Python 3, only `opencv-python` and `numpy`, CPU only, no model files besides small digit templates in the repo).

- `tracker = SudokuTracker()`: one object per camera stream. It keeps state between frames (the puzzle read so far), so a server needs one tracker per user session.
- `result = tracker.process_frame(frame)`: `frame` is a BGR `uint8` numpy array ($H \times W \times 3$), any size. Synchronous, not thread-safe.
- `draw_overlay(frame, result, n)`: returns a copy of the frame with the first `n` cells of `result["order"]` drawn on the paper. The app decides when to increase `n`.
- `tracker.reset()`: forget the current puzzle (e.g. a "new puzzle" button).

`result` is a plain dict of lists, ints, floats and strings, so it can be sent as JSON:

```
{"status": "solved",            # no_grid | reading | no_solution | solved
 "corners": [[x,y], ...],       # 4 floats, frame pixels, order TL, TR, BR, BL
 "homography": [[...], ...],    # 3x3, square (0..450) -> frame
 "givens":   [[5,3,0,...], ...],  # 9x9, printed digits, 0 = empty
 "solution": [[5,3,4,...], ...],  # 9x9, null unless status is solved
 "order": [[r,c], ...]}           # empty cells in reveal order, r,c in 0..8
```

- Coordinates: x = column, y = row, origin top-left of the frame. Rows and columns of the puzzle are 0–8 from the top-left of the puzzle.
- Status meaning for the app: `no_grid` = no puzzle in view; `reading` = puzzle found, hold still; `no_solution` = the digits read make the puzzle unsolvable; `solved` = start or continue the reveal.
- Reveal order: the cell with the fewest candidates comes first, like a person would solve it, so a single step also works as a hint.
- If the app prefers to draw by itself (e.g. in a browser canvas), it can draw each digit at the cell center in the 450×450 square and map it with `homography`.

A stub with the same interface and fake data comes first, so the app can be built in parallel.

3. Pipeline

1. **Grid detection:** grayscale, Gaussian blur, adaptive threshold (handles shadows), largest contour that approximates to 4 corners. Alternative to compare: Canny edges + Hough lines, keep the outermost horizontal and vertical lines, intersect them.

2. **Rectification:** homography from the 4 corners to a 450×450 square.
3. **Cells:** split into 81 cells of 50×50, cut the borders to remove grid lines. A cell is empty if the ink pixels in its center are below a threshold.
4. **Digit reading (classical):** the largest ink blob is centered and resized, then compared by normalized cross-correlation (template matching) to digits 1–9 rendered from a few common fonts.
5. **Solver (by hand):** check the givens for duplicates in rows, columns and boxes, then backtracking that fills the cell with the fewest candidates first. Also gives the reveal order.
6. **Overlay:** digits drawn on a transparent 450×450 layer, warped back to the frame with the inverse homography and blended.

4. Live behaviour

- Reading and solving run once, when the grid has been found in a few consecutive frames with little corner motion. The result is cached.
- Every frame only re-detects the 4 corners and smooths them over time (moving average) to reduce jitter, then redraws the overlay.
- If the grid is lost for a short time, the last result is kept. If it is lost for longer, or the digits read differ from the cache, the tracker resets.
- Targets on a laptop CPU at 640×480: about 20 FPS for the per-frame path, under 0.5 s for reading + solving. Large frames are downscaled internally.

5. Assumptions and limits

- One puzzle in view, roughly upright (rotated less than about 45°), filling a good part of the frame. Standard 9×9 puzzles with printed digits.
- Curved paper and strong glare are tested and reported, not guaranteed.

6. Evaluation (for the report)

Our own recorded videos and photos: flat, tilted, shadows, glare, curved paper, different fonts. A few frames per video labelled by hand (corners and digits). Metrics: grid detection rate (contour vs Hough), digit accuracy per cell, fully solved rate, FPS, overlay jitter (corner motion while the paper is still).

7. Repo and working rules

- My code is in `sudoku_cv/`: `detect.py`, `digits.py`, `solver.py`, `overlay.py`, `tracker.py` and `templates/`. `eval/` holds the evaluation scripts. The app lives in its own folder. We do not edit each other's folders; interface changes are agreed first.
- Sample frames are committed in `samples/` so the app can be tested without a camera.

8. Timeline

Shared dates for coordination:

- **Oct 28:** interface stub pushed (fake data).
- **Nov 15:** real pipeline behind the same interface, working live on a webcam.
- **Nov 16 – 22:** integration of the app with the real pipeline.
- **Nov 30:** code freeze.
- **Dec 1/3:** presentation. **Dec 11:** report.